Handing In Instructions updated October 19. Crack the Code typo fixed October 21

ENG 1P13: Computing Assignment 2

The Big Heist

In this assignment, you will create functions for a text-based game in which the player attempts to "break in" to a secure facility. Some of the details of the output are up to you (e.g. are they breaking into a bank, an underwater bunker, a moon base, etc.). The game will consist of four scenes, each of which is implemented by a different Python **function**. Then a fifth function will call the others in order and display the results.

The scenes are: 1. Enter; 2. Search a Room; 3. Crack a 3-digit Code; 4. Escape.

For this assignment, you are only allowed to use the features of Python that were shown to you in the Computing 1 to 5 labs and lectures. All the tools you need to do this assignment can be found there. Use nesting, Boolean expressions, and/or helper functions to keep the code as short and clean as possible. Do not use any global variables inside your functions.

Enter (first scene)



Write a function **enter(choice, code)** that accepts two parameters:

- **choice:** An integer from 1 to 6 representing the player's chosen method of entry.*
- **code:** The three-digit integer code (see below).

The function **must use random.randint(1,6)** exactly three times to simulate guards patrolling the building. If a guard's number is higher than **choice**, a guard notices something.

The function should also determine a clue. If **choice** 4 or greater, the clue is that the code is in the range 100 to 499 or 500 to 999. Otherwise, the clue should be the first (leftmost) digit of the code.

Suspicion: Add one point of suspicion for every guard that suspects something.

Print: A **single line** message containing the count of how many guards were alerted.

Return: Two integers. The first is the amount of **suspicion**, the second is the **clue** (either the leftmost digit, or -1 for the low range clue, or -2 for the high range clue).

Examples:

- Choice = 4, Code = 231, Random Numbers = 6, 5, 1 → Return 2, -1
- Choice = 1, Code = 982, Random Numbers = 3, 3, 4 → Return 3, 9

---

* The lower the number of the choice, the riskier the entry method. The text for the options will be presented to the player in the **heist** method described below.

Search a Side Room (Second Scene)

Write a function **side_room(investigate, code)** that accepts two parameters**:**

- **investigate:** Boolean value **True** or **False**.
- **code:** The three-digit integer code (see below).

If **investigate** is **True**, the function generates a random integer (you **must use random.randint(1,100)** for this). If the integer is in the range 81 to 100, the user finds the entire code. If the integer is in the range 31 to 80, they find the third (rightmost) digit of the code. If **investigate** is false, no suspicion should be gained, and no clue should be given.

Suspicion: Use the value of the random integer you generated, divided by 20 and rounded up.

Print: If **investigate** is True, print on **one line** a description of the room, otherwise print on **one line** a message that they skipped the room.

Return: Two integers. The first is the amount of suspicion generated in this scene, the second the clue they received (or -1 if no clue).

Examples:

- Investigate = True, Code = 138 random integer = 85 → Return 5, 138
- Investigate = False, Code = 821, no random integer generated → Return 0, -1

Crack the Code (Third Scene)

Write a function **crack_code(suspicion, code)** that accepts two parameters**:**

- **suspicion:** An integer representing suspicion generated so far.
- **code:** A three-digit integer code (see below).



The function asks the user to guess until either they run out of attempts or guess correctly. The maximum number of attempts is **10 – suspicion**.

Suspicion: Add 1 for an **incorrect** guess plus 3 more if they fail to guess the code.

Print: Print on **one line** the number of guesses they get (e.g. “You have time for 5 attempts.”). Then prompt on **one line** for a guess and print on **one line** a full report of each attempt to crack the safe (e.g. “Attempt 3: Guess = 232, incorrect”). For example, if the user gets 3 guesses, this function will at most output 7 lines (one for the number of guesses and a prompt and report line for each guess).

Return: An integer and a Boolean. The integer is the amount of **new suspicion** (e.g. if they came in with 3 suspicion and generated 4 suspicion trying to crack the safe, the function should return 4), followed by **True** if they cracked the code or **False** otherwise.

Examples:

- Suspicion = 4, code = 851, 6 guesses, cracked on 3rd guess → **Return 2**, True
- Suspicion = 0, code = 125, 10 guesses, not cracked → Return 13, False

Escape[†] (final scene)

Write a function called **escape(suspicion)** that accepts one integer parameter:

- **suspicion:** The total accumulated suspicion so far

The player begins with **danger = suspicion // 3** and **progress = 0**. Repeatedly prompt the player to choose an action. For each action use **random.randint(1, 100)** to generate a random integer:

- **Run**: add 1 **progress** if random integer is 60 or less, add 1 **danger** and 1 **progress** otherwise
- **Hide**: add 1 **progress** if random integer is 80 or less, add 2 **danger** otherwise
- **Shortcut**: add 3 **progress** if random integer is 50 or less, add 3 **danger** otherwise

The escape ends when either **progress** or **danger** reaches 10 or more. (If progress reaches 10 or more, they escape. Otherwise, if danger reaches 10 or more, they're caught.)

Print: Prompt on **one line** to ask the player which action (e.g., "Run, Hide, or Shortcut?") and print on **one line** a full report of each action (e.g., "You hide but they find you! Progress = 4, Danger = 7").

Return: The final integer **progress** value**.**

Heist[‡]

Create a function called **heist (name)** that controls the entire game. The function should:

- Greet the player by name and inform them of the mission
- Generate a random three-digit integer code and store it in a local variable. Use **random.randint(100, 999)** for this. If the global constant **DEBUG** is **True**, print the code.
- **Call** the function for each scene in the order given above.
    - **Input** required information before calling a function (e.g. show them the **entry** methods, ask if they want to search the **side room**, etc.).
    - **Print** an interpretation of any **return value** (e.g., **progress** returned 12 so they made it).
    - If they find a **clue**, print a message telling them what the clue was.
    - Track and display cumulative **suspicion**.
- Output introductions, progress, and success/failure messages as appropriate. The exact level of detail or amount of text is up to you but you should output enough text between function calls to make it feel like a text-based adventure game.

Trying again: If the player is not successful (they got caught or they escaped without cracking the code) ask whether they want to try again. If they choose yes, restart the game from the beginning, reinitializing all the variables. If the player escapes after cracking the code, display a success message and end the game.

---

† This function may require techniques that you will learn in Computing 5 (Indefinite Iteration).
‡ This function may also require techniques that you will learn in Computing 5 (Indefinite Iteration).

The Main Program

When the program is run, it should first set the global constant **DEBUG** to **True**, then call a **print_signature** function that behaves as described in **Computing 3 Exercise 1**. Then it should ask the user for their name and call the **heist** function (name can be stored in a global variable).

Programming to Specifications

On the job, it's often important that your code meet certain rigid specifications. We will run automated test cases on each of the functions to generate part of your grade. It is very important that your functions be named correctly, accept the correct number and order of parameters, return the correct values in the correct order, respect the output limits (number of lines printed – no extra lines, blank lines, or lines with **\n** characters) and make the correct calls to **random.randint**. It is possible that your code could be logically correct but still fail test cases because you have not followed the specifications correctly.

The Check In

You will get time to work on this in the next lab. During that time, the teaching staff will come to check up on how you are doing. They will want to see you've made progress and are not leaving everything to the last minute. They may ask you questions about your work.

Before the next lab, you should AT LEAST have **function stubs** for each scene, your **print_signature** function, and a **heist** function that runs a single iteration.

Documentation

Follow the documentation standards in **documentation_standards.py** on Avenue.

Extra Challenges (Optional)

If you want to add more scenes to the game, feel free to add more, each in its own function, as long as the win/lose conditions of the game do not change (e.g. 10 risk = caught, 10 progress = success). But make sure that you include the four functions described above. Feel free to add turtle graphics and sound as well, although for ease of testing, we ask that you stick to shell input and output for this assignment (i.e., don't use the Turtle graphics input methods)

Handing In

Rename your python program **MacID_ComputingAssignment2.py** (using your own MacID) and hand it in to the **Computing Assignment 2** area on Avenue. Then upload a second version of the file but name this one **MacID_ComputingAssignment2.txt**. This is to make the file easier for automated tools to scan.

Evaluation

Your program will be evaluated according to the rubric attached to Assignment 2 in Avenue.

Coding Advice

- “Get something working and keep improving it” (From textbook Chapter 3)
- “Code the interface first (Function Stub), then fill in the details.” (From textbook Chapter 6)
- Code a few lines, test, repeat.
- See the **Four Golden Rules** in the **Testing Supplement** on Avenue and make sure you test thoroughly, because we will!!
- For debugging:
    - Print the code at the top of the screen so you know what it is when you’re testing.
    - Use print statements to trace which code blocks are executing and to display the values of variable as the program runs.
    - Remember to remove these debugging print statements before handing in.

Step 1: Write Function Stubs

Put “stubs” in place of the “scenes”. These function stubs should have the correct function interface but just have a single print statement in the body and return default values. For example, just print “Entry code goes here” and return a appropriate place holder values.

Step 2: Write and Debug One Iteration of **heist**

Before creating the full game, write a simple version of **heist** that generates a code for the safe, interacts with the user, calls each function once, tracks suspicion, and displays an interpretation of the returned values. At this stage, the game does not need to be fully functional. The goal is to make sure all functions are communicating correctly.

Step 3: Complete and Test Each Scene Individually

Implement the game one scene at a time. Test each function directly in the shell before connecting it to the rest of the game.

Step 4: Complete **heist** and the main program

Expand the **heist** function to include a loop that will allow the user to play until they succeed or quit trying. Then fill in the **main program** if you haven’t already.

Step 5: Improve the Player Experience

Once the game is fully functional, consider improving mission descriptions, output formatting, error messages, and story elements. Remember that a game that is easy to understand is much easier to test and debug.

Testing Suggestions

- Every entry method
- Both side room choices
- Correct and incorrect code guesses
- Successful and unsuccessful escapes
- Invalid menu choices

- Etc…

Following the **Four Golden Rules of Testing** will help you find bugs before we do!

Advice on Academic Integrity

Using code that you did not write yourself is a violation of academic integrity.

- At this stage in your programming career, getting code from a web site, online forum, friend, tutor, or AI assistant can only hurt your learning. It's easy to look at a piece of code and fool yourself into thinking that just because you "get it", you could have written it yourself. There's no substitute for doing your own coding when you're learning and using code from other sources puts you at risk of an academic integrity charge.
- If you need help with your code, use the time in lab and lecture, the office hours, and the CAS drop in help center to get help.

Protect Yourself: don't share your code with anybody else.

- If you can offer help to others, you should. Just don't send them your code or dictate code to them. Help them by explaining concepts, making suggestions, testing your code, helping them debug, etc.
- If you send your code to somebody else, you have no control over how they will use it. Even your best friend could make a bad decision at the 11th hour. If someone includes your code in their assignment, this will be considered a breach of academic integrity for both of you.